A PROJECT REPORT ON

EMPLOYEE ATTENDENCE AND MANAGEMENT SYSTEM USING FACE RECOGNITION

AREPORT SUBMITTED IN PARTIAL FULFILMENT

OF THE REQUERMENTS FOR THE AWARD OF THE DEGREE OF

BACHELOR OF COMPUTER APPLICATIONS

SUBMITTED BY

SOORYATHEJUS : (U19TP23S0002)

UNDER THE GUIDENCE OF

DR.K.Nithiyanandhan

(Professor)

VVIMS
VIJAYA VITTALA
INSTITUTE OF MANAGEMENT AND SCIENCES

DEPARTMENT OF COMPUTER APPLICATIONS

VIJAYA VITTALA INSTITUTE OF MANAGEMENT AND SCIENCE

BENGALURU, 560077

BATCH 2023 – 2026

DECLARATION

I hereby declare that the project titled "EMPLOYEE ATTENDENCE AND MANAGEMENT SYSTEM " is based on an original work of independent research, carried out by me in partial fulfilment of Bachelor of Computer Application under Bengaluru North University under the guidance of Dr.K.Nithiyanandhan

I also declare that this project is the outcome of my own efforts and that it has not been submitted to any other university or Institute for the award of any other degree or Diploma or Certificate in Bengaluru North University or any other universities.

PLACE: BENGALURU

DATE:

SOORYATHEJUS K

(REG. No: U19TP23S0002)

ACKNOWLEDGEMENT

It is my pleasure to acknowledge with thanks the people who guided me to complete this project successfully. I proudly utilize the opportunity to express my heart full thanks to the DR.K. NITHIYANANDHAN(Principal) Sit. Department of Computer Science Management for their valuable advice and encouragement for carrying out this project.

My special thanks to faculty guide DR.K.NITHIYANANDHAN, Department of Computer Science Management, for her supportive and excellent guidance and also comments during the progress of this research project. Her enthusiasm and encouragement for this project had helped me to a great extent towards completing the project.

Furthermore, I would like to appreciate all the Faculty Members in Department of Management and Computer Science Management for their continuous support and assistance during the progress of research.

Last but not least, my greatest gratitude to my family members and my friends on the moral support and the stimulation for their encouragement and support to compete the project work successfully.

SOORYATHEJUS K

REG. NO: U19TP23S0002

PLACE: BENGALURU

TABLE OF CONTENTS

SL.NO	TOPICS	PAGE NO
1	INTRODUCTION	1
2	LITERATURE SURVEY	5
3	SYSTEM ANALYSIS	7
4	SYSTEM REQUIREMENT	10
5	SYSTEM DESIGN	11
6	CODING	18
7	TESTING	68
8	RESULT	74
9	CONCLUSION	79
10	FUTURE ENHANCEMENT	80
11	BIBLIOGRAPHY	81

1. INTRODUCTION

The rise of e-commerce has not only reshaped consumer shopping habits but also redefined the competitive landscape for businesses. In fashion retail especially, the digital marketplace has become a hub of creativity, diversity, and accessibility. Customers are no longer restricted to what is available in their local stores; instead, they can explore global catalogs, compare styles, and discover emerging brands that align with their personal identity. This shift has empowered consumers to make more informed choices while encouraging businesses to innovate constantly in order to meet evolving expectations.

A clothing web application serves as the backbone of this transformation, offering a centralized platform where buyers and sellers interact seamlessly. For customers, the experience is enriched through advanced search filters, detailed product descriptions, and interactive features such as size charts and customer reviews. These elements reduce uncertainty and build confidence in online purchases. The inclusion of wish lists, personalized recommendations, and order tracking further enhances convenience, creating a shopping journey that feels both intuitive and reliable. Secure payment gateways and multiple transaction options ensure that trust remains at the core of the digital retail experience.

On the seller's side, the platform simplifies complex retail operations. Product uploads, inventory management, and pricing adjustments can be handled with ease, while integrated analytics provide valuable insights into customer behavior. These insights allow businesses to tailor marketing campaigns, optimize stock levels, and introduce promotions that resonate with their audience. For small businesses and startups, the reduced overhead costs compared to physical stores make it possible to compete with established brands on a global scale, levelling the playing field in ways that were previously unimaginable.

Technically, the success of such applications depends on robust architecture and modern design principles. Database-driven systems ensure that product catalogs remain dynamic and scalable, while responsive design guarantees accessibility across devices. Frontend technologies like HTML, CSS, and JavaScript create engaging interfaces, while backend frameworks handle transactions, authentication, and data security. Cloud hosting and microservices architecture further strengthen scalability, ensuring that the platform can handle peak traffic during seasonal sales or promotional events without compromising performance. Security measures such as encryption, fraud detection, and compliance with international standards safeguard both customer and business interests. Looking ahead, the future of fashion e-commerce is poised to become even more immersive and personalized.

1.1 Aim

The aim of the E-commerce Cloth Selling Online Web Application project is to design and develop a modern digital platform that allows clothing businesses to showcase and sell their products online while providing customers with a convenient and efficient shopping experience. The system focuses on creating a user-friendly interface where customers can easily browse different categories of clothing, view product details, select sizes, and place orders through a secure online environment.

The main objective of the application is to simplify the buying and selling process by connecting customers and sellers through a centralized web platform. Customers can explore a wide range of clothing products, compare styles and prices, add items to a shopping cart, and complete purchases using secure payment methods. The platform is designed to provide a smooth and responsive experience across different devices, ensuring accessibility anytime and anywhere.

1.2 Objectives

- To develop a user-friendly online platform for selling and purchasing clothing products.
- To allow customers to browse different categories of clothes and view product details such as size, color, and price.
- To provide a shopping cart system that enables customers to add and manage items before purchasing.
- To design a smooth, responsive, and attractive interface that improves the overall online shopping experience.
- To enable customers to place orders securely through an online system that protects user information and supports safe transactions.
- To allow customers to track their orders and receive purchase updates.
- To provide an administrative system for sellers or administrator**s** to manage clothing products efficiently.
- To allow administrators to update inventory, modify product prices, and monitor customer orders**.**
- To demonstrate the use of modern web technologies, database management, responsive design, and security mechanisms in building a scalable and efficient clothing e-commerce platform.

1.3 Methodology

System Requirement Gathering

- Collect functional and non-functional requirements
- Identify user requirements and administrator requirements
- Analyze system feasibility and workflow
- Define hardware and software specifications

Functional Requirements

- User registration and login
- Product management
- Shopping cart management
- Order placement and tracking
- Admin dashboard management

Non-Functional Requirements

- Security
- Performance
- Scalability
- Reliability
- Responsive user interface

System Design and Architecture

- Create system architecture diagrams
- Design database schema and relationships
- Define module interactions and workflow
- Plan API structure and authentication flow

Frontend Development

- Develop user interface using TSX and CSS
- Implement responsive design using Tailwind CSS / Bootstrap
- Create product catalog and shopping cart pages
- Integrate authentication pages
- Develop reusable frontend components
- Ensure mobile-friendly user experience

Backend Development

- Implement server-side logic
- Connect application with MySQL/PostgreSQL database
- Develop REST APIs for products and orders
- Integrate Firebase authentication and synchronization
- Manage user sessions and authentication
- Handle product and order data securely

2. LITERATURE SURVEY

1. Consumer Behaviour in Online Apparel Shopping – Reddy (2025)

Study:

This paper analyses how factors like gender, discounts, convenience, and delivery influence online clothing purchases. It uses survey data (100 respondents) to understand buying patterns in apparel e-commerce.

Limitation:

- Very small sample size (only 100 people) → results are not reliable for large populations.
- Focus limited to one region (Hyderabad), so it cannot be generalized globally.

2. Web-Based Clothes Sales Information System (2025)

Study:

This research focuses on developing a web-based clothing sales system for a specific shop, including product management and sales processing.

Limitation:

- Designed for a single store, not scalable for large e-commerce platforms.
- Lacks advanced features like recommendation systems or secure payment gateways.

3. Effect of Fashion E-Commerce User Experience – Ofori (2024)

Study:

This paper studies how website design, navigation, and service quality affect customer retention in fashion e-commerce.

Limitation:

- Based on secondary data (desk research), not real-time experiments.
- Does not provide system development or coding details.

4. Consumer Buying Behaviour on Fashion Wears (2025)

Study:

This research examines how social, cultural, and personal factors influence clothing purchase decisions in online platforms.

Limitation:

- Focuses more on psychology than actual e-commerce system design.
- Does not address technical challenges like security or scalability.

5. E-Commerce Application in Fashion Shop (Research Study)

Study:

:This paper discusses the implementation of an e-commerce system for fashion stores, focusing on business processes and online transactions.

Limitation:

- Lacks detailed technical architecture (no clear backend/frontend explanation).
- Does not include modern technologies like cloud or mobile optimization.

6. Recommendation System for Fashion E-Commerce – Hwangbo et al. (2018)

Study:

This research develops a recommendation system for fashion products using customer preferences and seasonal trends.

Limitation:

- Requires large datasets and advanced algorithms → not suitable for small projects.
- Complex to implement in basic web applications.

3. SYSTEM ANALYSIS

System analysis is a critical phase in the development of any software project, as it helps identify business requirements, understand user needs, and design an effective solution. In the case of the Online Clothing E-Commerce System, the analysis focuses on addressing the challenges faced by traditional clothing retail businesses and providing a convenient digital shopping experience for customers.

Traditional clothing stores require customers to visit physical locations, which can be time-consuming and may limit product accessibility. Managing inventory, tracking sales, handling customer orders, and maintaining accurate records manually can also lead to errors and inefficiencies. Furthermore, customers often face difficulties in comparing products, checking availability, and accessing a wide variety of fashion items in one place.

The Online Clothing E-Commerce System is designed to overcome these challenges by providing a centralized web-based platform where customers can browse, search, and purchase clothing products from anywhere at any time. The system enables users to create accounts, view product catalogs, add items to shopping carts, place orders, and make secure online payments. It also provides administrators with tools to manage products, categories, inventory, customer information, and order processing efficiently.

From a functional perspective, the system includes modules for user registration and authentication, product management, shopping cart management, order processing, payment integration, and customer support. Customers can easily search for products, filter items based on categories, sizes, colors, and prices, and track their orders after purchase. Administrators can monitor sales, update product details, manage stock levels, and generate business reports.

Non-functional requirements such as security, performance, scalability, reliability, and usability play a vital role in ensuring the success of the system. The platform must protect customer data and payment information, provide fast response times, support a growing number of users and products, and offer an intuitive user interface for seamless navigation.

The architecture of the system typically consists of a presentation layer (user interface), an application layer (business logic), a database layer (storage of product, customer, and order information), and a payment gateway integration layer. Together, these components ensure efficient operation, secure transactions, and a smooth online shopping experience for customers while simplifying business management for administrators.

2.1 Existing System

The existing clothing retail system is primarily based on traditional physical stores where customers must visit the shop to browse and purchase products. Product information, inventory management, and sales records are often maintained manually or through basic computerized

systems. This approach requires significant human effort and can lead to errors in stock management, billing, and record keeping.

Customers have limited access to products as shopping is restricted to store operating hours and geographical location. Comparing products, checking availability, and finding desired clothing items can be time-consuming. Additionally, traditional systems lack real-time inventory updates, making it difficult for customers and store managers to know the exact availability of products.

Order processing, inventory tracking, and customer management are often handled separately, resulting in inefficiencies and delays. These limitations highlight the need for an integrated online platform that provides convenient shopping, automated inventory management, secure transactions, and efficient order processing.

Disadvantages:

- Limited Accessibility
- Time-Consuming Process
- Poor Inventory Management
- Limited Product Availability
- Higher Operational Costs
- Lack of Real-Time Updates

2.2 Proposed System

The proposed system is an Online Clothing E-Commerce Website that allows customers to browse, search, and purchase clothing products through a web-based platform. The system provides product catalogs, shopping cart functionality, secure online payments, and order management. It enables customers to shop anytime and from anywhere while allowing administrators to manage products, inventory, and customer orders efficiently.

Advantages:

• Easy Online Shopping

- 24/7 Product Availability
- Secure Online Transactions
- Efficient Inventory Management
- Faster Order Processing
- Better Customer Experience

4. HARDWARE AND SOFTWARE REQUIREMENTS

2.3 Hardware Requirements

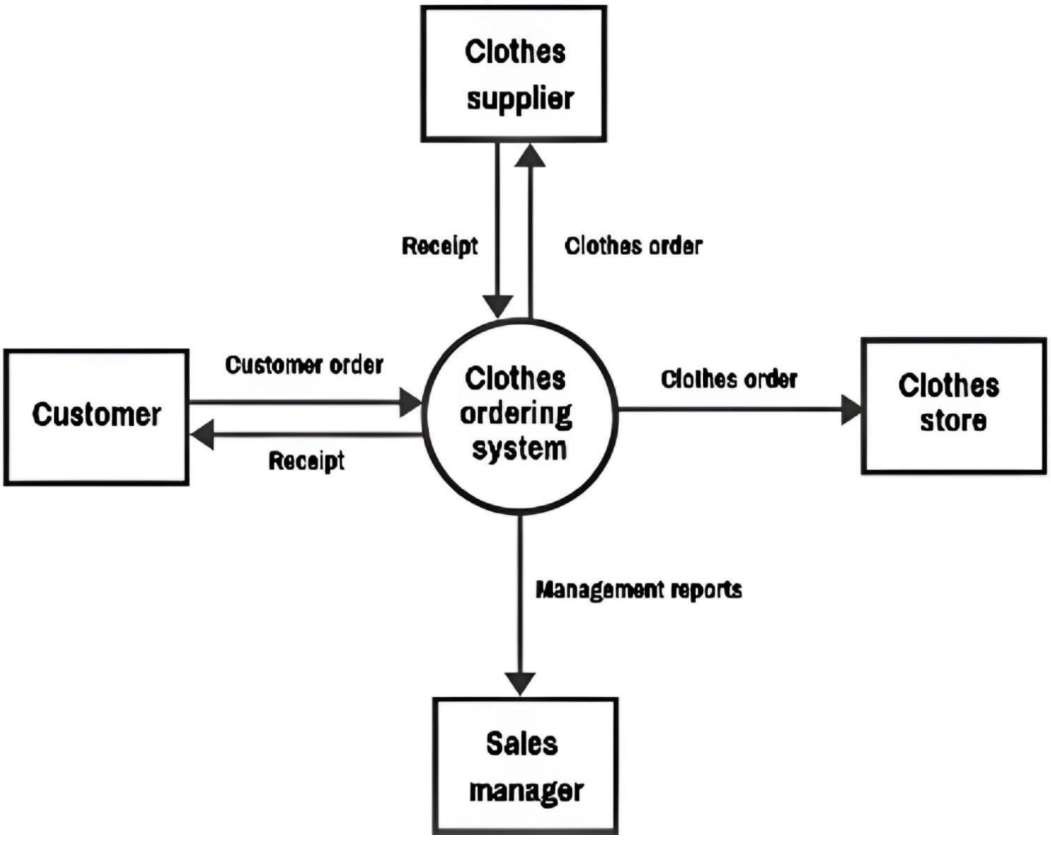
- Processor: Intel Core i5
- RAM: 16 GB
- Storage: 500 GB SSD
- Display: Full HD (1920 × 1080)
- Network: High-speed broadband
- Server (if self-hosted): Quad-core CPU, 16 GB RAM, 1 TB SSD

2.4 Software Requirements

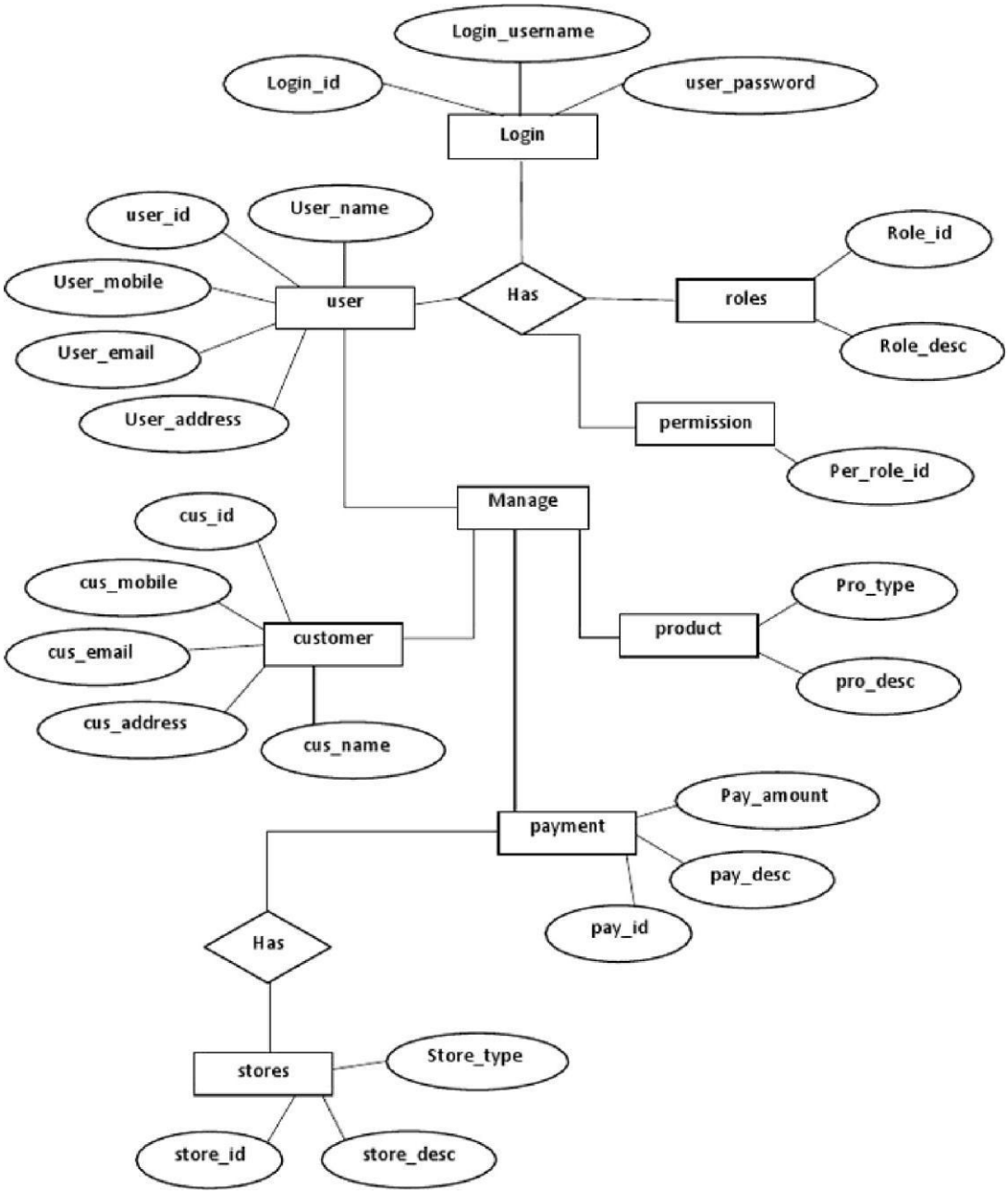
- Operating System: Windows 11
- Languages: HTML, CSS, JavaScript, PHP, Python, Java
- Database: MySQL / Django
- IDE/Tools: Visual Studio Code, Eclipse, IntelliJ IDEA
- Version Control: Git with GitHub/GitLab

5. SYSTEM DESIGN

2.5 DFD Diagram



2.6 Entity Relationship (ER) Diagram



2.7 Database Design

Users Table

Field	Type	Null	Default
User_ID	Int	No	No
Name	Varchar(100)	No	No
Email	Varchar(100)	No	No
Password	Varchar(255)	No	No
Phone	Varchar(15)	Yes	Null
Address	Varchar(255)	Yes	Null

Admin Table

Field	Type	Null	Default
Admin_ID	Int	No	No
Name	Varchar(100)	No	No
Email	Varchar(100)	No	No
Password	Varchar(255)	No	No

Product Table

Field	Type	Null	Default
Product_ID	Int	No	No
Product_Name	Varchar(150)	No	No
Category	Varchar(50)	No	No
Price	Decimal(10,2)	No	No
Stock	Int	No	No
Description	Text	Yes	Null
Image_Path	Varchar(255)	Yes	Null

Category Table

Field	Type	Null	Default
Category_ID	Int	No	No
Category_Name	Varchar(50)	No	No

Cart Table

Field	Type	Null	Default
Cart_ID	Int	No	No
User_ID	Int	No	No
Product_ID	Int	No	No
Quantity	Int	No	1

Orders Table

Field	Type	Null	Default
Order_ID	Int	No	No
User_ID	Int	No	No
Order_Date	DateTime	No	No
Total_Amount	Decimal(10,2)	No	No
Order_Status	Varchar(50)	No	Pending

Order Details Table

Field	Type	Null	Default
Order_Detail_ID	Int	No	No
Order_ID	Int	No	No
Product_ID	Int	No	No
Quantity	Int	No	No
Price	Decimal(10,2)	No	No

Payment Table

Field	Type	Null	Default
Payment_ID	Int	No	No
Order_ID	Int	No	No
Payment_Method	Varchar(50)	No	No
Payment_Status	Varchar(50)	No	Pending
Payment_Date	DateTime	Yes	Null

Wishlist Table

Field	Type	Null	Default
Wishlist_ID	Int	No	No
User_ID	Int	No	No
Product_ID	Int	No	No

6. MODULES

To ensure structured development and maintainability, the system is divided into several functional modules.

1. **Product Catalog Module**

   The Product Catalog Module serves as the primary interface between the store and its customers. It displays all available clothing products in an organized, visually appealing grid layout, allowing users to browse items effortlessly. Products are categorized into sections such as Men's, Women's, Shoes, and Apparel, enabling fast and intuitive navigation. Each product listing includes high-resolution images, a concise product name, pricing in Indian Rupees, available size variants, and a brief description highlighting key features. The module supports dynamic filtering, allowing customers to narrow results by category or search keyword in real time. On the individual product detail page, users can view an expanded description, select their preferred size from a dropdown menu, check stock availability, and add the item directly to their cart. The catalog is powered by a live database (MySQL via Sequelize ORM), ensuring that product updates made by administrators — such as price changes, stock adjustments, or new product additions — are immediately reflected on the storefront without requiring a page rebuild. The module also handles edge cases such as displaying an "Out of Stock" label and disabling the Add to Cart button when a product's stock reaches zero, preventing invalid orders. On the technical side, the catalog leverages Next.js server-side rendering for fast initial page loads and improved search engine visibility, while client-side state management handles real-time interactions such as size selection and cart additions without full page reloads.

2. **Shopping Cart Module**

   The Shopping Cart Module is a central component of the user's purchasing journey, bridging product selection and order confirmation. When a customer clicks "Add to Cart" on any product detail page, the selected item — along with its chosen size and quantity — is added to a persistent cart managed through React's Context API (CartContext). This global state ensures that cart contents are accessible across all pages, including the header badge that displays the live item count. The cart data is serialized and stored in the browser's localStorage, meaning items persist even if the user refreshes the page or navigates away, providing a seamless and uninterrupted shopping experience. On the dedicated Cart page, customers can view a detailed summary of all selected items, each showing the product image, name, selected size, unit price, quantity controls (increment and decrement buttons), and line-item subtotal. Users can adjust quantities in real time, and the cart total automatically recalculates

with each change. Removing an item is a single-click action via the delete icon, and if the cart becomes empty, a friendly empty-state message is displayed with a prompt to continue shopping. A persistent Order Summary panel on the right side of the cart page displays the current subtotal and a note that shipping is calculated at checkout. The "Proceed to Checkout" button transitions the user to the checkout page, carrying all cart data forward. Edge cases such as adding duplicate items in different sizes are handled by treating each size as a unique cart entry, while the same product in the same size increments the existing quantity rather than creating a duplicate row.

3. Order Management Module

The Order Management Module handles the complete lifecycle of a customer's purchase from the moment checkout is initiated through to final payment confirmation. When a customer submits their shipping details on the Checkout page, the module first validates authentication by checking for a valid JWT token in localStorage, redirecting unauthenticated users to the login page before any transaction can proceed. Once authentication is confirmed, the module sends a POST request to the backend orders API, passing the cart items, shipping details, and calculated total. The backend then creates a Razorpay payment order and returns a unique order ID and the payable amount in paisa. This triggers the Razorpay payment modal, which is loaded via a dynamically injected script, presenting the customer with a secure payment interface pre-filled with their name, email, and order details. Upon successful payment, Razorpay redirects the transaction to the backend's callback endpoint, where HMAC-SHA256 signature verification is performed to cryptographically confirm the authenticity of the payment. If the signature matches, the order status in the database is updated from "pending" to "paid", the cart is cleared, and the customer is redirected to the Order Confirmation page displaying a success message. Administrators can monitor all orders in the database, view customer shipping details, check payment IDs, verify Razorpay signature records, and update order statuses. The module also handles failure cases: if payment is declined or a network error occurs, the order retains a "failed" status and the user is shown a descriptive error message. All order data — including item lists, totals, and shipping information — is stored as structured JSON in the MySQL database, ensuring easy retrieval and auditability.

4. Authentication Module

The Authentication Module provides the security foundation for the entire Terra Fit platform, managing user identity verification for both customers and administrators. The module implements a stateless JWT (JSON Web Token) based authentication system, where tokens are issued upon successful login and stored in the browser's localStorage for subsequent authenticated requests. During user registration, the module accepts a full name, email address, and password. Passwords are never stored in plain text; instead, they are hashed using bcrypt with a salt factor of 10 before being saved to the MySQL database through the Sequelize ORM User model. This ensures that even in the event of a database breach, raw passwords remain protected. During login, the submitted password is compared against the stored hash using bcrypt's compare function, and if valid, a signed JWT containing the user's ID and admin flag is returned and stored client-side. All protected routes — such as checkout, order history, and admin operations — verify the presence and validity of this token through a middleware function that decodes and validates the JWT signature on every request. If the token is absent, expired, or tampered with, the middleware rejects the request with a 401 Unauthorized response, and the frontend redirects the user to the login page. The admin flag within the token enables role-based access control, allowing only users with isAdmin: true to access administrative endpoints such as product creation, deletion, and order status updates. The module also exposes a logout mechanism that removes the token and user data from localStorage, effectively ending the session on the client side.

5. Cloud Synchronization Module

The Cloud Synchronization Module extends the platform's capabilities beyond local infrastructure by integrating cloud-based services to ensure data reliability, scalability, and real-time accessibility. At its core, this module leverages Firebase — Google's Backend-as-a-Service platform — to provide supplementary authentication mechanisms and real-time data synchronization for critical application data including product listings and order records. Firebase Authentication serves as a secondary identity provider, supporting email/password-based sign-in alongside potential future expansion to social login providers such as Google and Facebook. Real-time synchronization via Firebase Realtime Database or Firestore ensures that any update made to product inventory, pricing, or order status is instantly propagated across all connected clients without requiring a manual page refresh, delivering a live and responsive experience to administrators monitoring the dashboard. The module also supports cloud-hosted media assets through integration with services such as Cloudinary or Firebase Storage, enabling product images to be stored and served efficiently from globally distributed CDN nodes rather than the local server's

filesystem. This reduces server load and dramatically improves image load times for users across different geographic locations. Additionally, the module is designed with future scalability in mind: as the platform grows, database workloads can be offloaded progressively to cloud infrastructure (MongoDB Atlas or PlanetScale for MySQL), reducing dependency on self-hosted servers and enabling automated backups, point-in-time recovery, and horizontal scaling without application downtime. The cloud synchronization layer therefore acts as a bridge between the local development environment and a fully distributed production architecture, ensuring that Terra Fit can scale from a college project to a commercially viable platform with minimal re-engineering.

3. CODING

Home page:

```
"use client";


import React, { useState, useEffect, useCallback } from "react";

import Link from "next/link";

import Image from "next/image";

import styles from "./page.module.css";

import { getProducts, Product } from "@/data/products";


// ── Hero slides
──────────────────────────────────────────────────────────────────
──────────────

const HERO_SLIDES = [

  {

    id: 1,

    image:

      "https://images.unsplash.com/photo-1542291026-
7eec264c27ff?q=80&w=1800&auto=format&fit=crop",

    tag: "NEW SEASON DROP",

    headline: "Built For The",

    accent: "Streets.",

    sub: "Ultra-performance footwear for every terrain.",

    cta: "Shop Shoes",

    href: "/products?category=Shoes",
```

```
    align: "left",
  },
  {
    id: 2,
    image:
      "https://images.unsplash.com/photo-1515886657613-
9f3515b0c78f?q=80&w=1800&auto=format&fit=crop",
    tag: "WOMEN'S EDIT",
    headline: "Own Your",
    accent: "Look.",
    sub: "Discover premium activewear crafted for women.",
    cta: "Shop Women",
    href: "/products?category=Women's",
    align: "right",
  },
  {
    id: 3,
    image:
      "https://images.unsplash.com/photo-1441984904996-
e0b6ba687e04?q=80&w=1800&auto=format&fit=crop",
    tag: "ALL GEAR",
    headline: "Gear Up.",
    accent: "Stand Out.",
    sub: "Performance apparel, footwear & accessories.",
    cta: "Explore All",
    href: "/products",
```

```
    align: "left",
  },
];

// ── Announcement items ──────────────────────────────────────────────

const ANNOUNCEMENTS = [
  "□  Free delivery on orders above ₹999",
  "□  Shop Men's — New Arrivals Just Dropped",
  "□  New season arrivals — Shop Now",
  "□  Easy 7-day returns",
  "□  100% Secure Payments",
  "⚡  Limited time: Extra 10% off with code TERRAFIT10",
];

export default function Home() {
  const [products, setProducts] = useState<Product[]>([]);
  const [slideIndex, setSlideIndex] = useState(0);

  // fetch products
  useEffect(() => {
    getProducts().then((p) => setProducts(p.slice(0, 8)));
  }, []);

  // auto-advance hero carousel
```

```
  const next = useCallback(
    () => setSlideIndex((i) => (i + 1) % HERO_SLIDES.length),
    []
  );
  const prev = () =>
    setSlideIndex((i) => (i - 1 + HERO_SLIDES.length) % HERO_SLIDES.length);

  useEffect(() => {
    const t = setInterval(next, 4500);
    return () => clearInterval(t);
  }, [next]);

  const slide = HERO_SLIDES[slideIndex];

  return (
    <div className={styles.page}>
      {/* ── Announcement Bar ──────────────────────────────────────────────── */}
      <div className={styles.announcementBar}>
        <div className={styles.announcementTrack}>
          {[...ANNOUNCEMENTS, ...ANNOUNCEMENTS].map((txt, i) => (
            <span key={i} className={styles.announcementItem}>
              {txt}
            </span>
          ))}
```

```
      </div>
    </div>

    {/* ── Hero Carousel ──────────────────────────────────────────────────────── */}
    <section className={styles.hero}>
      {HERO_SLIDES.map((s, idx) => (
        <div
          key={s.id}
          className={`${styles.heroSlide} ${
            idx === slideIndex ? styles.heroSlideActive : ""
          }`}
        >
          <Image
            src={s.image}
            alt={s.headline}
            fill
            priority={idx === 0}
            className={styles.heroImg}
            sizes="100vw"
          />
          <div className={styles.heroGrad} />
        </div>
      ))}
```

```
      {/* Content overlay */}
      <div
        className={`${styles.heroContent} ${
          slide.align === "right" ? styles.heroContentRight : ""
        }`}
      >
        <span className={styles.heroTag}>{slide.tag}</span>
        <h1 className={styles.heroHeadline}>
          {slide.headline} <span className={styles.heroAccent}>{slide.accent}</span>
        </h1>
        <p className={styles.heroSub}>{slide.sub}</p>
        <Link href={slide.href} className={styles.heroCta}>
          {slide.cta} →
        </Link>
      </div>

      {/* Prev / Next */}
      <button className={`${styles.heroArrow} ${styles.heroArrowLeft}`}
onClick={prev}>
        ‹
      </button>
      <button className={`${styles.heroArrow} ${styles.heroArrowRight}`}
onClick={next}>
        ›
      </button>
```

```
        {/* Dots */}
        <div className={styles.heroDots}>
          {HERO_SLIDES.map((_, i) => (
            <button
              key={i}
              className={`${styles.heroDot} ${i === slideIndex ? styles.heroDotActive : ""}`}
              onClick={() => setSlideIndex(i)}
            />
          ))}
        </div>
      </section>

        {/* ── Category Grid ──────────────────────────────────────────────────────── */}

      <section className={styles.section}>
        <div className={styles.container}>
          <h2 className={styles.sectionHeading}>SHOP BY CATEGORY</h2>
          <div className={styles.catGrid}>
            {[
              {
                label: "Men's",
                href: "/products?category=Men's",
                img: "https://images.unsplash.com/photo-1617127365659-
c47fa864d8bc?q=80&w=800&auto=format&fit=crop",
              },
              {
                label: "Women's",
```

```
            href: "/products?category=Women's",
            img: "https://images.unsplash.com/photo-1515886657613-
9f3515b0c78f?q=80&w=800&auto=format&fit=crop",
          },
          {
            label: "Shoes",
            href: "/products?category=Shoes",
            img: "https://images.unsplash.com/photo-1542291026-
7eec264c27ff?q=80&w=800&auto=format&fit=crop",
          },
          {
            label: "Apparel",
            href: "/products?category=Apparel",
            img: "https://images.unsplash.com/photo-1591047139829-
d91aecb6caea?q=80&w=800&auto=format&fit=crop",
          },
          {
            label: "All Gear",
            href: "/products",
            img: "https://images.unsplash.com/photo-1441984904996-
e0b6ba687e04?q=80&w=800&auto=format&fit=crop",
          },
        ].map((cat) => (
          <Link key={cat.label} href={cat.href} className={styles.catCard}>
            <Image
              src={cat.img}
              alt={cat.label}
```

```
              fill
              className={styles.catImg}
              sizes="(max-width: 768px) 50vw, 25vw"
            />
            <div className={styles.catOverlay} />
            <span className={styles.catLabel}>{cat.label}</span>
          </Link>
        ))}
      </div>
    </div>
  </section>

  {/* ── Promo Banner ──────────────────────────────────────────────────────── */}
  <section className={styles.promoBanner}>
    <Image
      src="https://images.unsplash.com/photo-1556906781-9a412961a28c?q=80&w=1800&auto=format&fit=crop"
      alt="New Season Drop"
      fill
      className={styles.promoImg}
      sizes="100vw"
    />
    <div className={styles.promoOverlay} />
    <div className={styles.promoContent}>
      <span className={styles.promoTag}>LIMITED EDITION</span>
```

```
        <h2 className={styles.promoHeadline}>New Season Drop</h2>
        <p className={styles.promoSub}>
          Fresh silhouettes. Bold palettes. Zero compromises.
        </p>
        <Link href="/products" className={styles.promoCta}>
          Explore Collection
        </Link>
      </div>
    </section>

    {/* ── Featured Products ──────────────────────────────────────────── */}
    <section className={styles.section}>
      <div className={styles.container}>
        <div className={styles.sectionRow}>
          <h2 className={styles.sectionHeading}>LATEST DROPS</h2>
          <Link href="/products" className={styles.viewAll}>
            View All →
          </Link>
        </div>
        <div className={styles.productsGrid}>
          {products.map((product) => (
            <Link
              key={product.id}
              href={`/products/${product.id}`}
```

```
            className={styles.productCard}
          >
            <div className={styles.productImgWrap}>
              <Image
                src={product.image}
                alt={product.name}
                fill
                className={styles.productImg}
                sizes="(max-width: 640px) 100vw, (max-width: 1024px) 50vw, 25vw"
              />
              <div className={styles.productBadge}>{product.category}</div>
              <div className={styles.productQuickAdd}>Quick Add</div>
            </div>
            <div className={styles.productInfo}>
              <p className={styles.productName}>{product.name}</p>
              <p className={styles.productPrice}>₹{product.price.toLocaleString("en-
IN")}</p>
            </div>
          </Link>
        ))}
      </div>
    </div>
  </section>
      {/* ── USP Strip
──────────────────────────────────────────────────────────────────────────
*/}
```

```
      <div className={styles.uspStrip}>
        <div className={styles.container}>
          <div className={styles.uspGrid}>
            {[
              { icon: "□", title: "Free Delivery", sub: "On orders above ₹999" },
              { icon: "□", title: "Easy Returns", sub: "7-day hassle-free returns" },
              { icon: "□", title: "Secure Payment", sub: "100% safe & encrypted" },
              { icon: "□", title: "Premium Quality", sub: "Crafted for performance" },
            ].map((u) => (
              <div key={u.title} className={styles.uspItem}>
                <span className={styles.uspIcon}>{u.icon}</span>
                <div>
                  <p className={styles.uspTitle}>{u.title}</p>
                  <p className={styles.uspSub}>{u.sub}</p>
                </div>
              </div>
            ))}
          </div>
        </div>
      </div>
    </div>
  );
}
```

Login page:

```
"use client";

import React, { useState } from "react";
import { useRouter } from "next/navigation";
import Link from "next/link";
import styles from "./page.module.css";

export default function LoginPage() {
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [showPassword, setShowPassword] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState("");
  const [success, setSuccess] = useState(false);

  const router = useRouter();

  const handleLogin = async (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    e.stopPropagation();

    if (!email || !password) {
      setError("Please fill in all fields.");
      return;
    }
```

```
  setIsLoading(true);
  setError("");
  setSuccess(false);

  const apiUrl = process.env.NEXT_PUBLIC_API_URL || "http://localhost:5000";

  try {
    const res = await fetch(`${apiUrl}/api/auth/login`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({ email, password }),
    });

    const data = await res.json();

    if (res.ok) {
      localStorage.setItem("token", data.token);
      localStorage.setItem("user", JSON.stringify(data.user));
      setSuccess(true);
      setTimeout(() => router.push("/"), 1000);
    } else {
      setError(data.error || "Login failed. Please check your credentials.");
    }
  } catch (err) {
```

```
      setError(`Cannot connect to server at ${apiUrl}. If using Render, it may be waking up—please wait 30 seconds and try again.`);
    } finally {
      setIsLoading(false);
    }
  };

  return (
    <div className={`container ${styles.page}`}>
      <div className={styles.loginCard}>
        <h1 className={styles.title}>Welcome Back</h1>
        <p className={styles.subtitle}>Log in to your Terra Fit account.</p>

        {error && <div className={styles.error}>⚠ {error}</div>}
        {success && <div className={styles.success}>✓Login successful! Redirecting...</div>}

        <form className={styles.form} onSubmit={handleLogin} method="post">
          <div className={styles.formGroup}>
            <label htmlFor="login-email">Email Address</label>
            <input
              type="email"
              id="login-email"
              name="email"
              value={email}
              onChange={(e) => setEmail(e.target.value)}
```

```
          required
          autoComplete="email"
          placeholder="you@example.com"
        />
      </div>

      <div className={styles.formGroup}>
        <label htmlFor="login-password">Password</label>
        <div className={styles.passwordWrapper}>
          <input
            type={showPassword ? "text" : "password"}
            id="login-password"
            name="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
            autoComplete="current-password"
            placeholder="••••••••"
          />
          <button
            type="button"
            className={styles.eyeButton}
            onClick={() => setShowPassword(!showPassword)}
            aria-label={showPassword ? "Hide password" : "Show password"}
          >
```

```
            {showPassword ? (
              <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
                <path d="M17.94 17.94A10.07 10.07 0 0 1 12 20c-7 0-11-8-11-8a18.45 18.45 0 0 1 5.06-5.94"/>
                <path d="M9.9 4.24A9.12 9.12 0 0 1 12 4c7 0 11 8 11 8a18.5 18.5 0 0 1-2.16 3.19"/>
                <line x1="1" y1="1" x2="23" y2="23"/>
              </svg>
            ) : (
              <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
                <path d="M1 12s4-8 11-8 11 8 11 8-4 8-11 8-11-8-11-8z"/>
                <circle cx="12" cy="12" r="3"/>
              </svg>
            )}
          </button>
        </div>
      </div>

      <button type="submit" className={styles.submitButton} disabled={isLoading || success}>
        {isLoading ? "Logging in..." : "Log In"}
      </button>
    </form>
      <p className={styles.switchText}>
```

```
          Don&amp;apos;t have an account?
          <Link href="/register" className={styles.switchLink}>Sign Up</Link>
        </p>
      </div>
    </div>
  );
}
```

Cart Page :

```
"use client";

import React from "react";
import Link from "next/link";
import Image from "next/image";
import { useCart } from "@/context/CartContext";
import styles from "./page.module.css";

export default function CartPage() {
  const { cart, updateQuantity, removeFromCart, cartTotal } = useCart();

  if (cart.length === 0) {
    return (
      <div className={`container ${styles.emptyCart}`}>
        <h1 className={styles.title}>YOUR CART IS EMPTY</h1>
        <p className={styles.subtitle}>Looks like you haven't added any gear yet.</p>
        <Link href="/products" className={styles.continueButton}>
          Start Shopping
        </Link>
      </div>
    );
  }

  return (
    <div className={`container ${styles.page}`}>
```

```
<h1 className={styles.title}>YOUR CART</h1>


<div className={styles.cartLayout}>
 <div className={styles.itemsList}>
  {cart.map((item) => (
   <div key={`${item.id}-${item.size}`} className={styles.cartItem}>
    <div className={styles.itemImageWrapper}>
     <Image
      src={item.image}
      alt={item.name}
      fill
      className={styles.itemImage}
      sizes="100px"
     />
    </div>

    <div className={styles.itemDetails}>
     <div className={styles.itemHeader}>
      <Link href={`/products/${item.id}`} className={styles.itemName}>
       {item.name}
      </Link>
      <button
       onClick={() => removeFromCart(item.id, item.size)}
       className={styles.removeButton}
       aria-label="Remove item"
```

```
            >

              <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20"
viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2"
strokeLinecap="round" strokeLinejoin="round"><path d="M3 6h18"></path><path d="M19
6v14c0 1-1 2-2 2H7c-1 0-2-1-2-2V6"></path><path d="M8 6V4c0-1 1-2 2-2h4c1 0 2 1 2
2v2"></path></svg>

            </button>

          </div>


          <div className={styles.itemAttributes}>

            <div className={styles.itemPrice}>₹{item.price.toFixed(2)}</div>

          </div>

          <div className={styles.itemSubtotal}>

            ₹{(item.price * item.quantity).toFixed(2)}

          </div>


          <div className={styles.quantityControls}>

            <button

              onClick={() => updateQuantity(item.id, item.size, Math.max(1, item.quantity -
1))}

              className={styles.qtyButton}

            >

              -

            </button>

            <span className={styles.qtyValue}>{item.quantity}</span>

            <button

              onClick={() => updateQuantity(item.id, item.size, item.quantity + 1)}
```

```
                  className={styles.qtyButton}
                >
                  +
                </button>
              </div>
            </div>
          </div>
        ))}
      </div>

      <div className={styles.summarySection}>
        <div className={styles.summaryCard}>
          <h2 className={styles.summaryTitle}>ORDER SUMMARY</h2>

          <div className={styles.summaryRow}>
            <span>Subtotal</span>
            <span>₹{cartTotal.toFixed(2)}</span>
          </div>
          <div className={styles.summaryRow}>
            <span>Shipping</span>
            <span>Calculated at checkout</span>
          </div>

          <div className={styles.summaryTotal}>
            <span>Total</span>
```

```
            <span>₹{cartTotal.toFixed(2)}</span>
          </div>

          <Link href="/checkout" className={styles.checkoutButton}>
            Proceed to Checkout
          </Link>
        </div>
      </div>
    </div>
  );
}
```

Checkout page :

```
"use client";

import React, { useState } from "react";
import { useCart } from "@/context/CartContext";
import styles from "./page.module.css";
import Link from "next/link";
import { useRouter } from "next/navigation";

// Extend window object to include Razorpay
declare global {
  interface Window {
    Razorpay: any;
  }
}

export default function CheckoutPage() {
  const { cart, cartTotal, clearCart } = useCart();
  const [isSubmitting, setIsSubmitting] = useState(false);
  const [error, setError] = useState("");
  const router = useRouter();

  const loadScript = (src: string) => {
    return new Promise((resolve) => {
      const script = document.createElement("script");
      script.src = src;
```

```
    script.onload = () => {
      resolve(true);
    };
    script.onerror = () => {
      resolve(false);
    };
    document.body.appendChild(script);
  });
};

const handleSubmit = async (e: React.FormEvent<HTMLFormElement>) => {
  e.preventDefault();
  const form = e.currentTarget;
  setIsSubmitting(true);
  setError("");

  const token = localStorage.getItem('token');
  if (!token) {
    alert('Please login first!');
    router.push('/login');
    return;
  }

  if (cart.length === 0) {
    alert('Your cart is empty!');
```

```
    setIsSubmitting(false);
    return;
  }

  const formData = new FormData(form);
  const shippingDetails = Object.fromEntries(formData.entries());

  // 1. Load Razorpay Script
  const resScript = await loadScript("https://checkout.razorpay.com/v1/checkout.js");

  if (!resScript) {
    alert("Razorpay SDK failed to load. Are you online?");
    setIsSubmitting(false);
    return;
  }

  const apiUrl = process.env.NEXT_PUBLIC_API_URL || "http://localhost:5000";

  const finalTotal = Number(cartTotal);
  console.log(`Final Total being sent: ${finalTotal}`);

  try {
    // 2. Create Order on Backend
    const res = await fetch(`${apiUrl}/api/orders`, {
      method: "POST",
```

```
    headers: {
      "Content-Type": "application/json",
      "Authorization": `Bearer ${token}`
    },
    body: JSON.stringify({
      items: cart,
      shippingDetails,
      total: finalTotal
    }),
  });

  const data = await res.json();

  if (!res.ok) {
    throw new Error(data.error || "Failed to create order");
  }

  // 3. Initialize Razorpay Payment
  const options = {
    key: process.env.NEXT_PUBLIC_RAZORPAY_KEY_ID,
    amount: data.amount,
    currency: data.currency,
    name: "Terra Fit",
    description: "Purchase from Terra Fit store",
    image: "/logo.png",
```

```
        order_id: data.order_id,
        callback_url: `${apiUrl}/api/orders/callback`,
        redirect: true,
        prefill: {
          name: `${shippingDetails.firstName} ${shippingDetails.lastName}`,
          email: shippingDetails.email,
          contact: "9999999999",
        },
        theme: {
          color: "#000000",
        },
      };

      const paymentObject = new window.Razorpay(options);
      paymentObject.open();

    } catch (err: any) {
      console.error(err);
      setError(err.message || "An error occurred. Make sure the backend server is running.");
    } finally {
      setIsSubmitting(false);
    }
  };

  if (cart.length === 0) {
```

```
  return (
    <div className={`container ${styles.emptyState}`}>
      <h1>YOUR CART IS EMPTY</h1>
      <Link href="/products" className={styles.backButton}>Back to Shop</Link>
    </div>
  );
}


return (
  <div className={`container ${styles.page}`}>
    <h1 className={styles.title}>CHECKOUT</h1>

    <div className={styles.checkoutLayout}>
      <div className={styles.formSection}>
        <h2 className={styles.sectionTitle}>Shipping Details</h2>
        {error && <div className={styles.error}>{error}</div>}

        <form onSubmit={handleSubmit} className={styles.form}>
          <div className={styles.formRow}>
            <div className={styles.formGroup}>
              <label htmlFor="firstName">First Name</label>
              <input type="text" id="firstName" name="firstName" required />
            </div>
            <div className={styles.formGroup}>
              <label htmlFor="lastName">Last Name</label>
```

```
            <input type="text" id="lastName" name="lastName" required />
          </div>
        </div>
          <div className={styles.formGroup}>
          <label htmlFor="email">Email</label>
          <input type="email" id="email" name="email" required />
        </div>
          <div className={styles.formGroup}>
          <label htmlFor="address">Address</label>
          <input type="text" id="address" name="address" required />
        </div>

        <div className={styles.formRow}>
          <div className={styles.formGroup}>
            <label htmlFor="city">City</label>
            <input type="text" id="city" name="city" required />
          </div>
          <div className={styles.formGroup}>
            <label htmlFor="zip">ZIP Code</label>
            <input type="text" id="zip" name="zip" required />
          </div>
        </div>

        <button
          type="submit"
```

```
            className={styles.submitButton}
            disabled={isSubmitting}
          >
            {isSubmitting ? "PROCESSING..." : `PAY ₹${cartTotal.toFixed(2)} WITH RAZORPAY`}
          </button>
          <p className={styles.disclaimer}>
            Secure payment powered by Razorpay.
          </p>
        </form>
      </div>
      <div className={styles.summarySection}>
        <div className={styles.summaryCard}>
          <h2 className={styles.sectionTitle}>Order Summary</h2>
          <div className={styles.itemsList}>
            {cart.map(item => (
              <div key={`${item.id}-${item.size}`} className={styles.summaryItem}>
                <div className={styles.itemInfo}>
                  <span className={styles.itemName}>{item.name}</span>
                  <span className={styles.itemMeta}>
                    Qty: {item.quantity} {item.size ? `| Size: ${item.size}` : ''}
                  </span>
                </div>
                <span className={styles.itemPrice}>₹{(item.price * item.quantity).toFixed(2)}</span>
              </div>
```

```
            ))}
          </div>
          <div className={styles.summaryTotals}>
            <div className={styles.summaryRow}>
              <span>Subtotal</span>
              <span>₹{cartTotal.toFixed(2)}</span>
            </div>
            <div className={styles.summaryRow}>
              <span>Shipping</span>
              <span>Free</span>
            </div>
            <div className={styles.summaryTotal}>
              <span>Total</span>
              <span>₹{cartTotal.toFixed(2)}</span>
            </div>
          </div>
        </div>
      </div>
    </div>
  );
}
```

Contact Page :

```
import React from "react";
```

```
import styles from "./page.module.css";
import Link from "next/link";

export default function ContactPage() {
  return (
    <div className={styles.contactPage}>
      <div className={styles.meshContainer}>
        <div className={styles.mesh1}></div>
        <div className={styles.mesh2}></div>
      </div>

      <div className={`container ${styles.content}`}>
        <Link href="/" className={styles.backLink}>
          ← BACK TO HOME
        </Link>

        <h1 className={styles.title}>
          GET IN <span className={styles.neonText}>TOUCH.</span>
        </h1>
        <div className={styles.grid}>
        <div className={styles.contactInfo}>
          <p className={styles.lead}>
            Have a question about an order or just want to say hello? Our team is here to help.
          </p>
```

```
          <div className={styles.methods}>
            <div className={styles.method}>
              <h3>EMAIL</h3>
              <a
href="https://mail.google.com/mail/?view=cm&fs=1&to=terrafit7.business@gmail.com"
target="_blank" rel="noopener noreferrer">
                terrafit7.business@gmail.com
              </a>
            </div>
            <div className={styles.method}>
              <h3>WHATSAPP</h3>
              <a href="https://wa.me/919567232977" target="_blank" rel="noopener
noreferrer">
                +91 95672 32977
              </a>
            </div>
             <div className={styles.method}>
              <h3>INSTAGRAM</h3>
              <a href="https://www.instagram.com/sooryah_h/?_pwa=1#" target="_blank"
rel="noopener noreferrer">
                @sooryah__h
              </a>
            </div>
          </div>
        </div>
         <div className={styles.formContainer}>
          <form className={styles.form}>
```

```
            <div className={styles.formGroup}>
              <label>NAME</label>
              <input type="text" placeholder="Your name" />
            </div>
            <div className={styles.formGroup}>
              <label>EMAIL</label>
              <input type="email" placeholder="Your email" />
            </div>
            <div className={styles.formGroup}>
              <label>MESSAGE</label>
              <textarea rows={5} placeholder="How can we help?"></textarea>
            </div>
            <button type="button" className={styles.submitButton}>
              SEND MESSAGE
            </button>
          </form>
        </div>
      </div>
    </div>
  </div>
  );
}
```

FAQ Page :

```
import React from "react";
```

```
import styles from "./page.module.css";

import Link from "next/link";


export default function FAQPage() {

  const faqs = [

    {

      question: "What is Terra Fit?",

      answer: "Terra Fit is a premium activewear brand focused on merging high-performance gear with futuristic urban aesthetics (Cyber-Mesh, Neon-Glow)."

    },

    {

      question: "How long does shipping take?",

      answer: "Standard shipping typically takes 3-5 business days within the country. International shipping can take 7-14 business days."

    },

    {

      question: "What is your return policy?",

      answer: "We offer a 30-day return policy for all unworn and unwashed items in their original packaging."

    },

    {

      question: "Are the neon elements battery-powered?",

      answer: "Most of our gear uses passive reactive materials that glow under UV light or reflective materials. Some premium items do feature active LED integration with rechargeable batteries."

    },

    {
```

```
    question: "How do I track my order?",
    answer: "Once your order is shipped, you will receive an email with a tracking number and a link to track your package."
  }
];

return (
  <div className={styles.faqPage}>
    <div className={styles.meshContainer}>
      <div className={styles.mesh1}></div>
      <div className={styles.mesh2}></div>
    </div>

    <div className={`container ${styles.content}`}>
      <Link href="/" className={styles.backLink}>
        ← BACK TO HOME
      </Link>

      <h1 className={styles.title}>
        FREQUENTLY ASKED <span className={styles.neonText}>QUESTIONS</span>
      </h1>

      <div className={styles.faqList}>
        {faqs.map((faq, index) => (
          <div key={index} className={styles.faqItem}>
```

```
            <h3 className={styles.question}>{faq.question}</h3>
            <p className={styles.answer}>{faq.answer}</p>
          </div>
        ))}
      </div>

      <div className={styles.contactPrompt}>
        <p>Still have questions?</p>
        <Link href="/contact" className={styles.contactButton}>
          CONTACT SUPPORT
        </Link>
      </div>
    </div>
  </div>
  );
}
```

User Register Page :

```
"use client";
```

```
import React, { useState } from "react";
import { useRouter } from "next/navigation";
import Link from "next/link";
import styles from "./page.module.css";

export default function RegisterPage() {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [showPassword, setShowPassword] = useState(false);
  const [isLoading, setIsLoading] = useState(false);
  const [error, setError] = useState("");
  const [success, setSuccess] = useState(false);

  const router = useRouter();

  const handleRegister = async (e: React.FormEvent<HTMLFormElement>) => {
    e.preventDefault();
    e.stopPropagation();

    if (!name || !email || !password) {
      setError("Please fill in all fields.");
      return;
    }
```

```
if (password.length < 6) {
  setError("Password must be at least 6 characters.");
  return;
}
setIsLoading(true);
setError("");
setSuccess(false);

const apiUrl = process.env.NEXT_PUBLIC_API_URL || "http://localhost:5000";

try {
  const res = await fetch(`${apiUrl}/api/auth/register`, {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ name, email, password }),
  });

  const data = await res.json();

  if (res.ok) {
    setSuccess(true);
    setTimeout(() => router.push("/login"), 1500);
  } else {
    setError(data.error || "Registration failed. Please try again.");
```

```
      }
    } catch (err) {
      setError(`Cannot connect to server at ${apiUrl}. If using Render, it may be waking up—please wait 30 seconds and try again.`);
    } finally {
      setIsLoading(false);
    }
  };
    return (
    <div className={`container ${styles.page}`}>
      <div className={styles.registerCard}>
        <h1 className={styles.title}>Join Terra Fit</h1>
        <p className={styles.subtitle}>Create your account to unlock premium gear.</p>
        {error && <div className={styles.error}>⚠ {error}</div>}
        {success && <div className={styles.success}>✓Registered! Redirecting to login...</div>}

        <form className={styles.form} onSubmit={handleRegister} method="post">
          <div className={styles.formGroup}>
            <label htmlFor="reg-name">Full Name</label>
            <input
              type="text"
              id="reg-name"
              name="name"
              value={name}
              onChange={(e) => setName(e.target.value)}
```

```
            required
            autoComplete="name"
            placeholder="John Doe"
          />
        </div>
          <div className={styles.formGroup}>
          <label htmlFor="reg-email">Email Address</label>
          <input
            type="email"
            id="reg-email"
            name="email"
            value={email}
            onChange={(e) => setEmail(e.target.value)}
            required
            autoComplete="email"
            placeholder="you@example.com"
          />
        </div>

        <div className={styles.formGroup}>
          <label htmlFor="reg-password">Password</label>
          <div className={styles.passwordWrapper}>
            <input
              type={showPassword ? "text" : "password"}
              id="reg-password"
```

```
            name="password"
            value={password}
            onChange={(e) => setPassword(e.target.value)}
            required
            autoComplete="new-password"
            placeholder="••••••••"
            minLength={6}
          />
          <button
            type="button"
            className={styles.eyeButton}
            onClick={() => setShowPassword(!showPassword)}
            aria-label={showPassword ? "Hide password" : "Show password"}
          >
            {showPassword ? (
              <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
                <path d="M17.94 17.94A10.07 10.07 0 0 1 12 20c-7 0-11-8-11-8a18.45 18.45 0 0 1 5.06-5.94"/>
                <path d="M9.9 4.24A9.12 9.12 0 0 1 12 4c7 0 11 8 11 8a18.5 18.5 0 0 1-2.16 3.19"/>
                <line x1="1" y1="1" x2="23" y2="23"/>
              </svg>
            ) : (
              <svg xmlns="http://www.w3.org/2000/svg" width="20" height="20" viewBox="0 0 24 24" fill="none" stroke="currentColor" strokeWidth="2" strokeLinecap="round" strokeLinejoin="round">
```

```
                <path d="M1 12s4-8 11-8 11 8 11 8-4 8-11 8-11-8-11-8z"/>
                <circle cx="12" cy="12" r="3"/>
              </svg>
            )}
          </button>
        </div>
      </div>
  <button type="submit" className={styles.submitButton} disabled={isLoading || success}>
          {isLoading ? "Creating Account..." : "Sign Up"}
        </button>
      </form>

      <p className={styles.switchText}>
        Already have an account?
        <Link href="/login" className={styles.switchLink}>Log In</Link>
      </p>
    </div>
  </div>
 );
}
```

Products Page :

```
import React from "react";
```

```
import Link from "next/link";
import ProductCard from "@/components/ProductCard";
import { getProducts } from "@/data/products";
import styles from "./page.module.css";

export default async function ProductsPage(
  props: { searchParams: Promise<{ category?: string; search?: string }> }
) {
  const searchParams = await props.searchParams;
  const currentCategory = searchParams.category || "All";
  const searchQuery = searchParams.search?.toLowerCase() || "";

  const allProducts = await getProducts();

  let filteredProducts = allProducts;

  // Filter by category - only if not searching
  if (currentCategory !== "All" && !searchQuery) {
    filteredProducts = filteredProducts.filter((p) => p.category === currentCategory);
  }

  // Filter by search query
  if (searchQuery) {
    filteredProducts = filteredProducts.filter(
      (p) =>
```

```
      p.name.toLowerCase().includes(searchQuery) ||
      p.description.toLowerCase().includes(searchQuery)
    );
  }

  const categories = ["All", "Shoes", "Apparel"];

  return (
    <div className={`container ${styles.page}`}>
      <div className={styles.header}>
        <h1 className={styles.title}>{currentCategory === "All" ? "All Gear" :
currentCategory}</h1>
      </div>

      <div className={styles.filters}>
        <div className={styles.filterList}>
          {categories.map((cat) => (
            <Link
              key={cat}
              href={cat === "All" ? "/products" : `/products?category=${cat}`}
              className={`${styles.filterButton} ${currentCategory === cat ? styles.active :
""}`}
            >
              {cat}
            </Link>
          ))}
```

```
        </div>
      </div>

      {filteredProducts.length > 0 ? (
        <div className={styles.productsGrid}>
          {filteredProducts.map((product) => (
            <ProductCard key={product.id} product={product} />
          ))}
        </div>
      ) : (
        <div className={styles.emptyState}>
          <p>No drops found for &quot;{searchParams.search || currentCategory}&quot;.</p>
          <Link href="/products" className={styles.clearSearch}>Clear all filters</Link>
        </div>
      )}
    </div>
  );
}
```

Orders Page :

```
"use client";
```

```
import React, { useEffect, useState } from "react";
import Link from "next/link";
import { useRouter } from "next/navigation";
import styles from "./page.module.css";
export default function OrdersPage() {
  const router = useRouter();
  const [isClient, setIsClient] = useState(false);
  useEffect(() => {
    setIsClient(true);
    // Simple check to make sure they are logged in to view orders
    const token = localStorage.getItem("token");
    if (!token) {
      router.push("/login");
    }
  }, [router]);
if (!isClient) return null;
   return (
    <div className={`container ${styles.page}`}>
      <div className={styles.successCard}>
        <div className={styles.iconWrapper}>
          <svg
            xmlns="http://www.w3.org/2000/svg"
            width="48"
            height="48"
            viewBox="0 0 24 24"
```

```
            fill="none"
            stroke="currentColor"
            strokeWidth="2"
            strokeLinecap="round"
            strokeLinejoin="round"
            className={styles.successIcon}
          >
            <path d="M22 11.08V12a10 10 0 1 1-5.93-9.14"></path>
            <polyline points="22 4 12 14.01 9 11.01"></polyline>
          </svg>
        </div>
        <h1 className={styles.title}>Order Confirmed!</h1>
        <p className={styles.message}>
          Thank you for shopping with Terra Fit. Your premium activewear will be processed shortly.
        </p>
        <Link href="/products" className={styles.continueButton}>
          Continue Shopping
        </Link>
      </div>
    </div>
  );
```

```
  return (
    <div className={`container ${styles.page}`}>
      <div className={styles.header}>
        <h1 className={styles.title}>{currentCategory === "All" ? "All Gear" :
currentCategory}</h1>
      </div>

      <div className={styles.filters}>
        <div className={styles.filterList}>
          {categories.map((cat) => (
            <Link
              key={cat}
              href={cat === "All" ? "/products" : `/products?category=${cat}`}
              className={`${styles.filterButton} ${currentCategory === cat ? styles.active :
```

7. TESTING

Testing is an essential phase in the development of the Terra Fit Premium Clothing Store,ensuring that all modules function correctly and meet the specified requirements. The process involved functional testing to validate user registration, product browsing, cart operations, and order processing, as well as non-functional testing to assess performance, responsiveness, and security. Unit testing was performed on individual backend API endpoints and frontend components, followed by integration testing to verify smooth interaction between the Next.js frontend and the Express REST API. System testing confirmed that the complete application— from product discovery through Razorpay payment verification — worked reliably under real-world conditions across multiple browsers and devices. Finally, user acceptance testing (UAT) ensured that the platform was visually appealing, intuitive to navigate, and met the expectations of its target streetwear audience. Through this comprehensive approach, testing guaranteed the system's accuracy, efficiency, and readiness for deployment.

3.1 Unit Testing

Unit Testing is the process of verifying individual modules of the system in isolation. In the Terra Fit Premium Clothing Store, modules such as user registration, user login, product listing, category filtering, cart operations, checkout, and order processing were tested separately using Postman for backend API endpoints and manual browser testing for frontend components. Each test case validated inputs, outputs, and error handling to ensure correctness— covering scenarios such as duplicate email registration, invalid login credentials, missing required fields, invalid product IDs, negative payment amounts, and unauthorised access to protected routes. The authentication module was tested with both valid and invalid JWT tokens to verify that protected routes correctly rejected unauthorised requests.

3.2 Integration Testing

Integration Testing was conducted to ensure that the different modules of the Terra Fit Premium Clothing Store worked together seamlessly. After unit testing validated individual components, they were combined and tested collectively. The focus was on verifying the data flow between the Next.js frontend and the Express REST API, as well as the proper communication between the backend and the MongoDB database. Key integration flows tested included user registration and login from the frontend form through to JWT token storage, product listing pages fetching and rendering live data from the database, cart items being correctly passed from the CartContext to the checkout page, and the complete Razorpay payment flow from order creation through HMAC signature verification to order confirmation. This stage helped identify interface errors and inconsistencies that occurred during the interaction between modules, such as incorrect API response formats, missing authorization headers, and CORS configuration issues. Integration testing confirmed that all modules communicated correctly and that the system delivered accurate and reliable results as a whole.

3.3 Test Cases

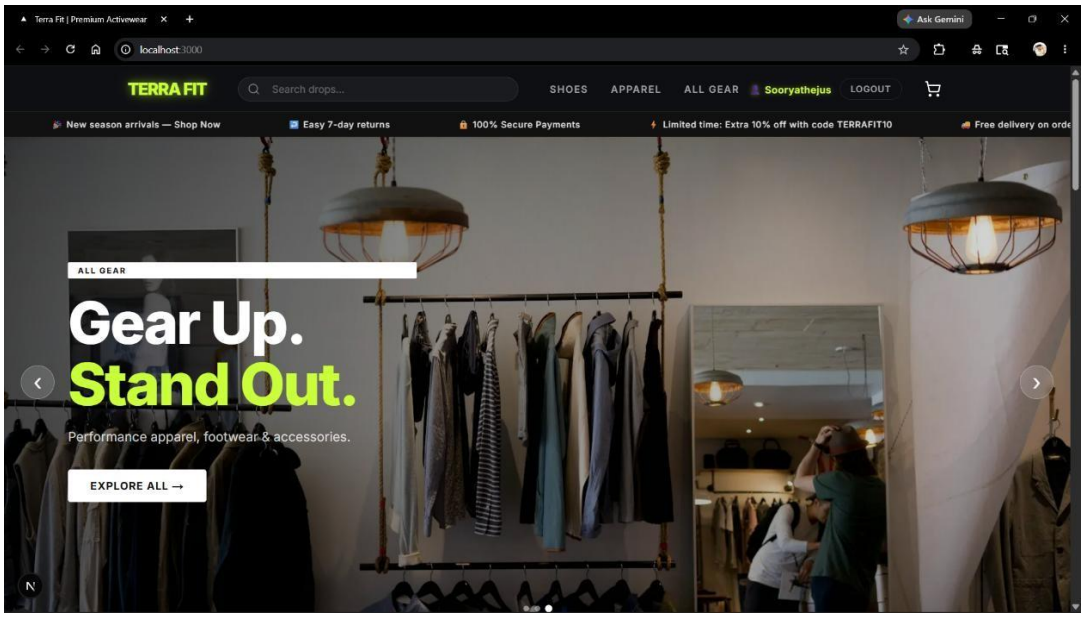
Test Case ID	Test Scenario	Steps	Expected Result	Actual Result
TC01	Verify application launch	Open http://localhost:3000 in browser	Homepage should load successfully with neon hero section	Homepage loaded successfully. Neon hero section rendered with animated gradient text and CTA buttons.
TC02	Verify page title	Open the application in browser	Page title "Terra Fit - Premium Streetwear" should display correctly	Browser tab displays "Terra Fit - Premium Streetwear" as expected.
TC03	Verify responsive UI	Open site on desktop, tablet, and mobile screen sizes	Layout should adjust properly across all screen sizes	Layout adapts correctly on desktop (1440px), tablet (768px), and mobile (375px). Hamburger menu appears on mobile.
TC04	Verify navigation links	Click all menu links (Home, Men, Women, All, Cart)	Each link should navigate to the correct page	All nav links functioned correctly. Home, Men, Women, All, and Cart routes navigated as expected.
TC05	Verify user registration	Enter valid name, email, and password and submit	Account should be created and JWT token stored successfully	Registration succeeded. JWT token stored in localStorage. User redirected to homepage.
TC06	Verify user login	Enter valid registered email and password and submit	User should be logged in and Navbar should show Orders and Logout	Login successful. Navbar updated to show "Orders" and "Logout" links. JWT token persisted.
TC07	Verify invalid login	Enter wrong email or incorrect password and submit	Error message "Invalid email or password"	Error message "Invalid email or password" displayed

			should display	correctly below the form on invalid credentials.
TC08	Verify mandatory field validation	Submit registration or login form with empty fields	Validation error messages should appear for required fields	Validation errors appeared for all empty required fields: Name, Email, and Password highlighted in red.
TC09	Verify product listing	Navigate to Products page	All products should load and display correctly in grid layout	All 12 products loaded and displayed in a 3-column responsive grid with image, name, and price.
TC10	Verify category filter	Click Men or Women filter button on Products page	Only products of the selected category should be displayed	Filter worked correctly. Clicking "Men" showed 6 men's products; "Women" showed 6 women's products.
TC11	Verify product detail page	Click on any product card	Full product details including name, price, description, and sizes should display	Product detail page loaded with full name, price (₹), description, available sizes, and Add to Cart button.
TC12	Verify Add to Cart	Click Add to Cart button on any product	Product should be added to cart and cart badge count should update	Product added to cart successfully. Cart icon badge updated from 0 to 1 with animation.
TC13	Verify cart persistence	Add items to cart and refresh the browser	Cart items should be restored from localStorage after page reload	Cart items persisted after browser refresh. All previously added items restored from localStorage.
TC14	Verify quantity update	Change item quantity in the Cart page	Item quantity and cart total should update correctly	Quantity increment/decrement worked. Total price recalculated correctly in real-time.
TC15	Verify remove from cart	Click Remove button on a cart item	Item should be removed and cart total should	Item removed successfully. Cart total recalculated

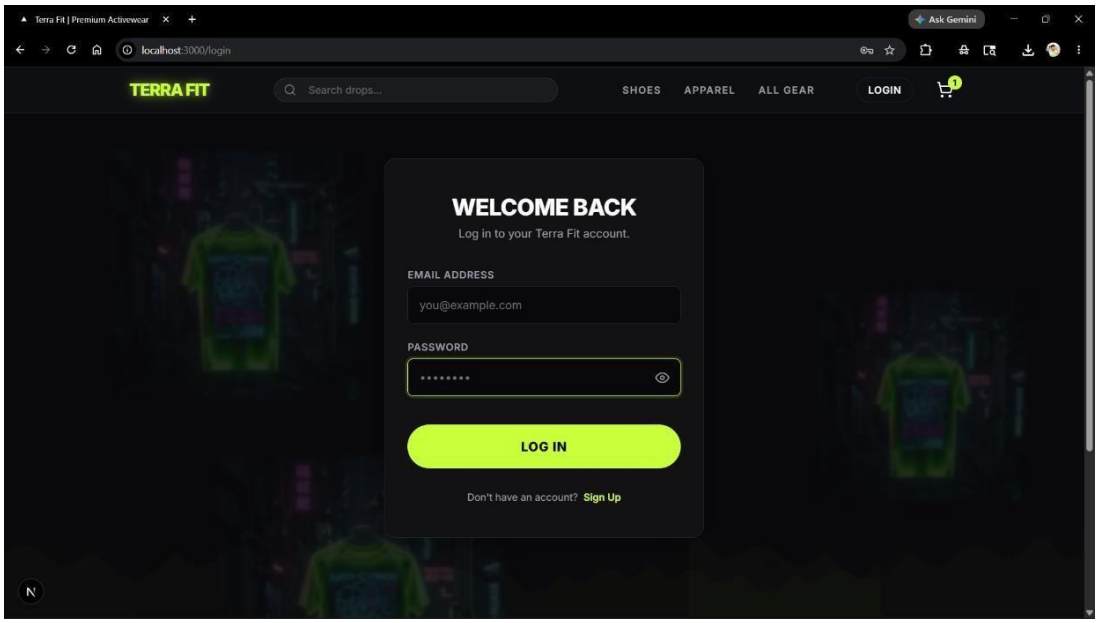
			recalculate	and cart badge count decremented.
TC16	Verify empty cart state	Remove all items from the cart	Empty cart message should be displayed	Empty cart state displayed correctly with message "Your cart is empty" and a Shop Now link.
TC17	Verify checkout form validation	Click Pay button without filling shipping address fields	Alert should appear asking user to fill all address fields	Alert box appeared: "Please fill in all address fields before proceeding." Payment was blocked.
TC18	Verify Razorpay payment modal	Fill valid address and click Pay button	Razorpay payment modal should open with correct order amount	Razorpay modal opened. Order amount matched cart total (₹). Test mode indicator visible.
TC19	Verify payment success flow	Complete test payment in Razorpay modal	Order should be saved with status paid and cart should be cleared	Payment completed. Order saved with status "paid" in DB. Cart cleared and user redirected to Orders page.
TC20	Verify order history	Navigate to Orders page after successful purchase	All past orders should be listed with correct details and status	Orders page listed all past orders with item names, quantities, amounts, and "paid" status correctly.
TC21	Verify protected route access	Try accessing checkout or orders page without login	User should be redirected to login page	Unauthenticated access to /checkout and /orders redirected to /login as expected.
TC22	Verify logout functionality	Click Logout button in Navbar	Token should be removed and Navbar should show Login and Register	JWT token removed from localStorage. Navbar reverted to showing "Login" and "Register" links.
TC23	Verify out of stock product	Open a product with stock set to zero	Add to Cart button should be disabled and show Out of Stock	Add to Cart button disabled and labeled "Out of Stock" in grey. Clicking produced no action.
TC24	Verify browser	Open site in Chrome, Firefox,	Application should work	Application functioned correctly

	compatibility	and Edge	correctly in all three browsers	in Chrome 124, Firefox 125, and Microsoft Edge 124. No rendering issues found.
TC25	Verify invalid input handling	Enter invalid email format during registration	Proper validation error message should display	Validation message "Please enter a valid email address" displayed for inputs like "user@" or "notanemail".

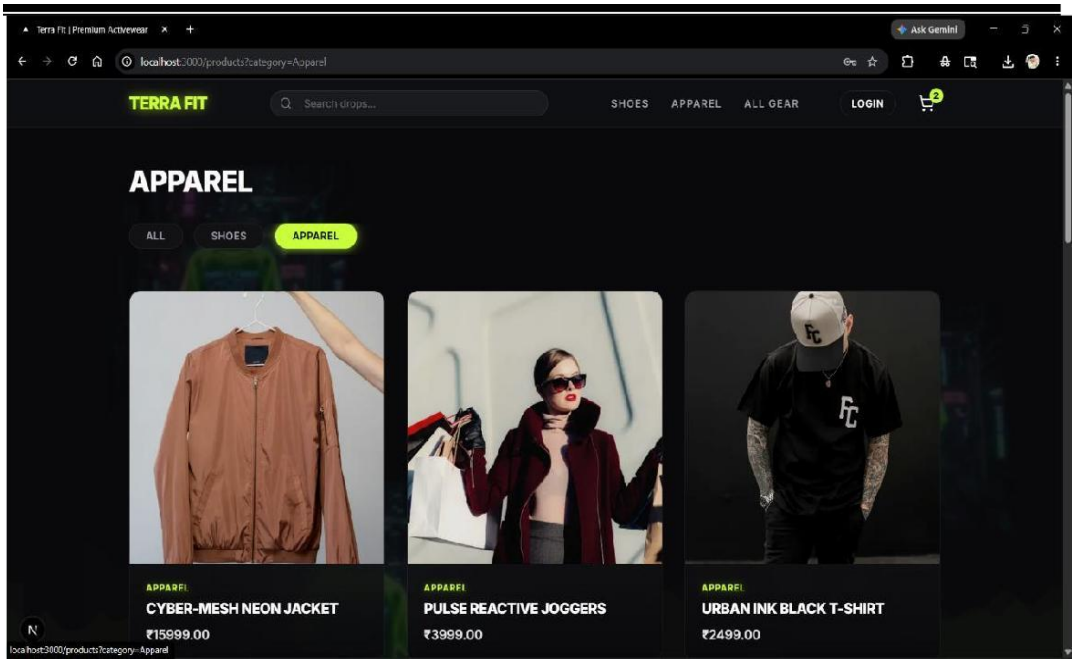
8. RESULT



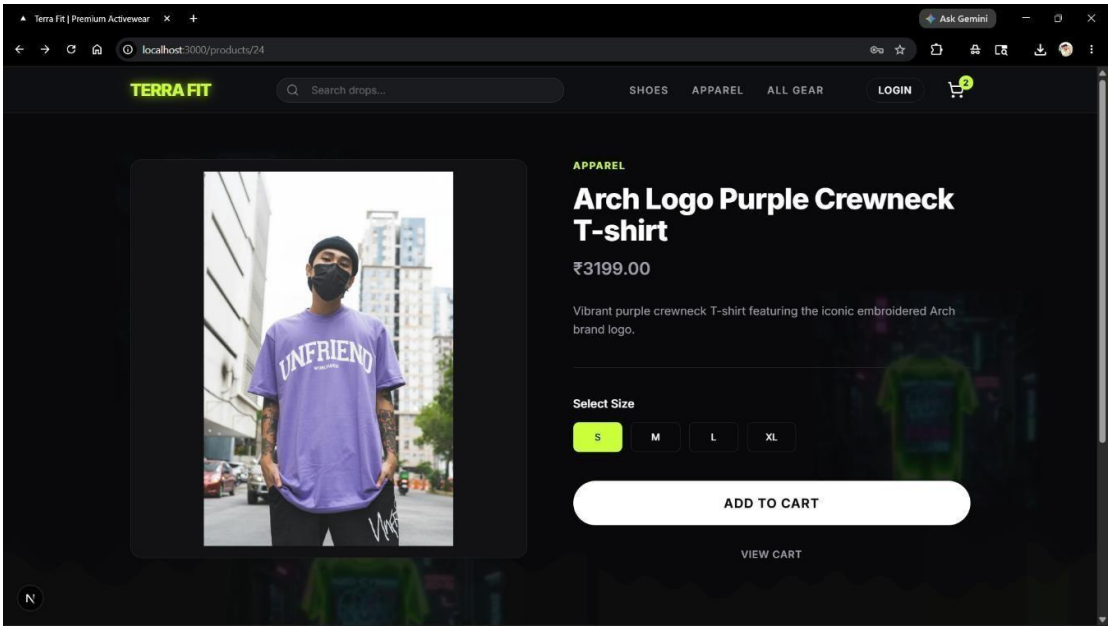
(fig:1-Home Page)



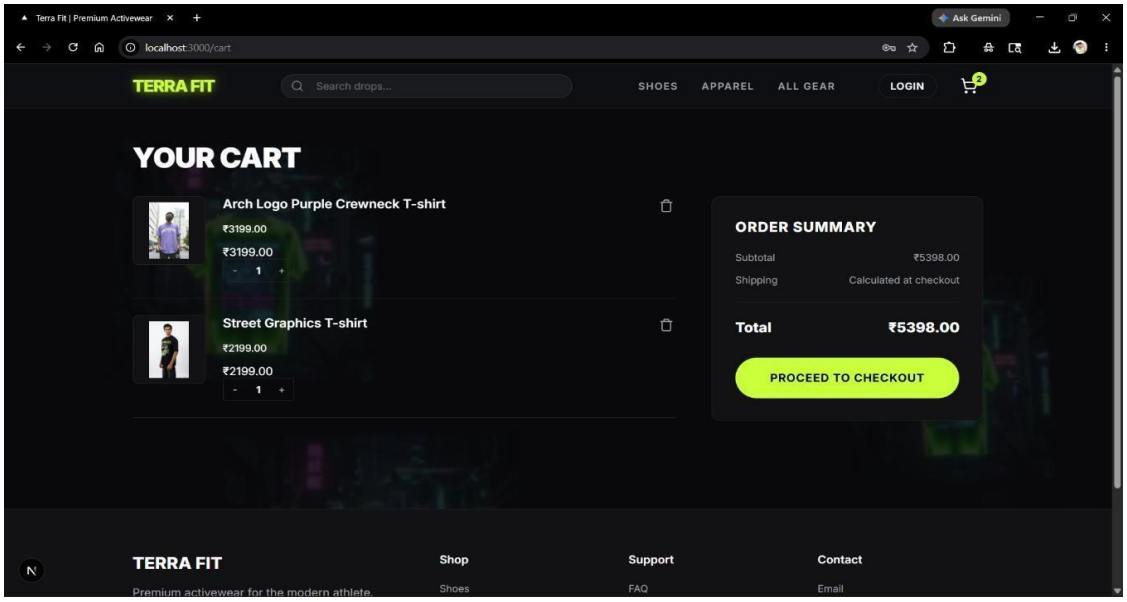
(fig:2- Login Page)



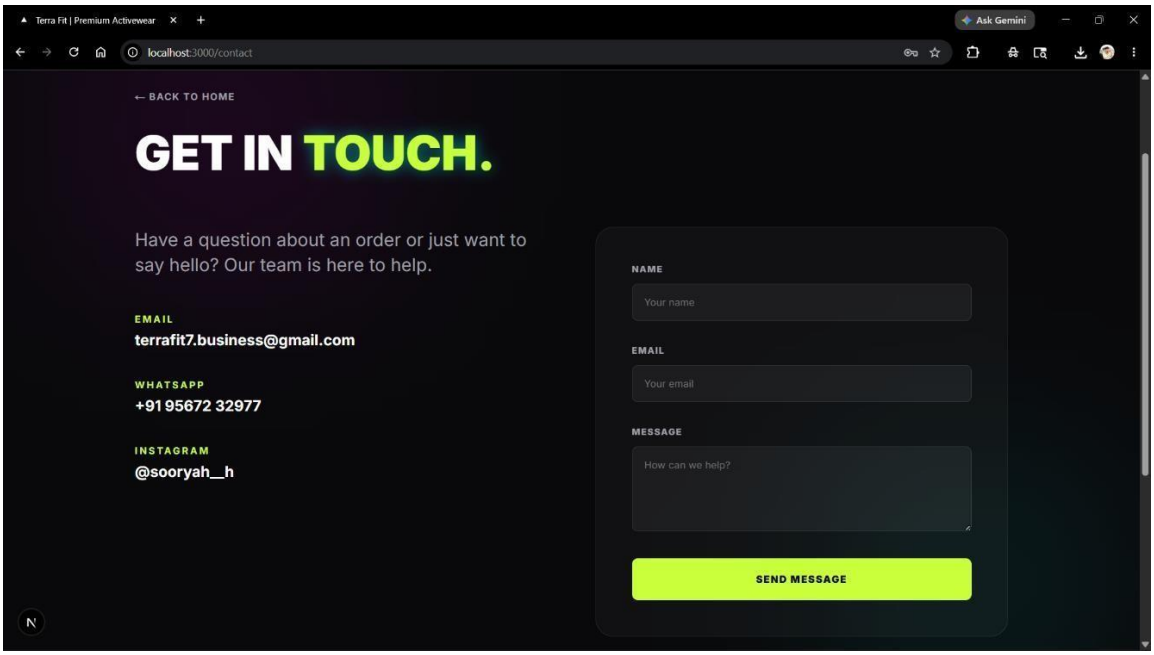
(fig:3- Products catalogue)



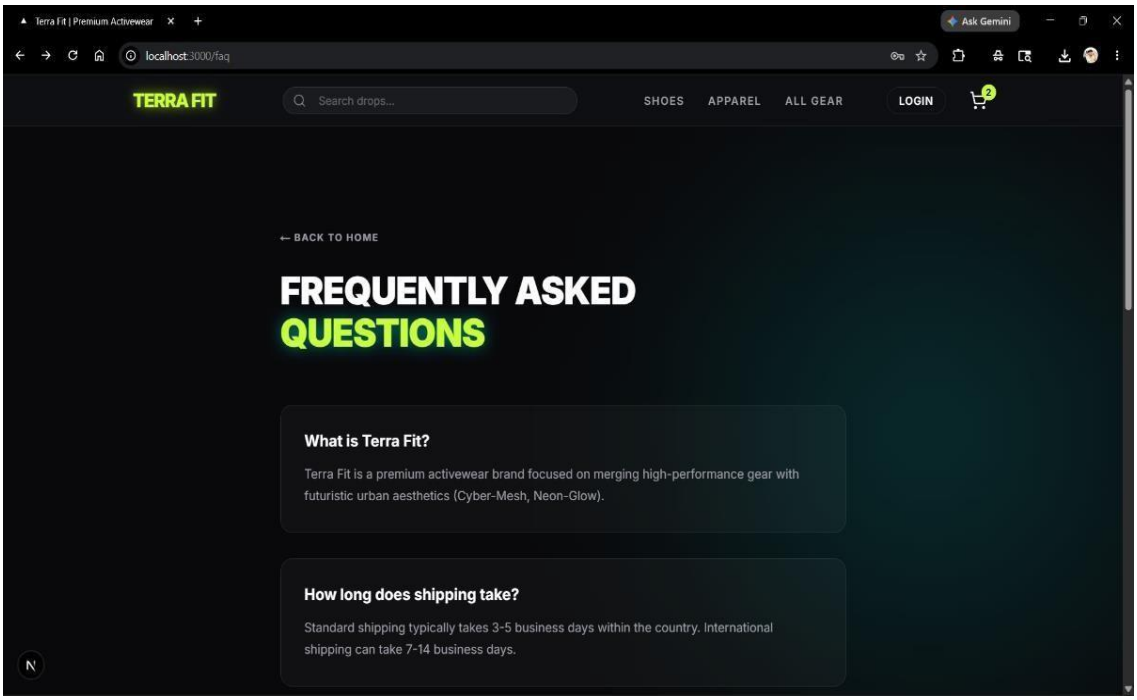
(fig:4- Product Details)



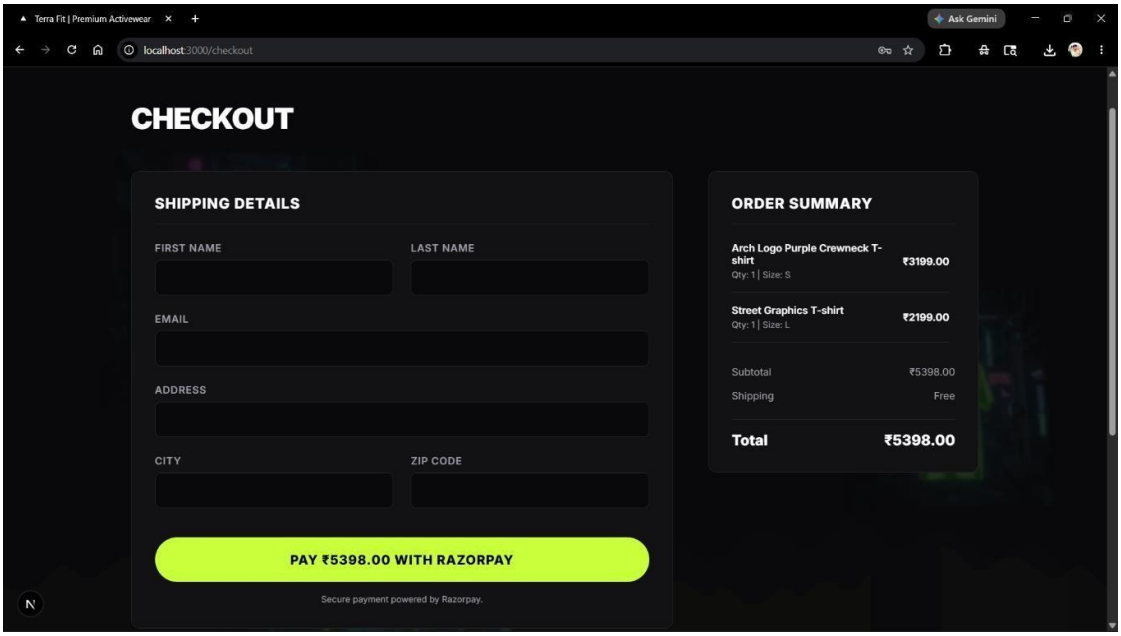
(fig:5- Cart)



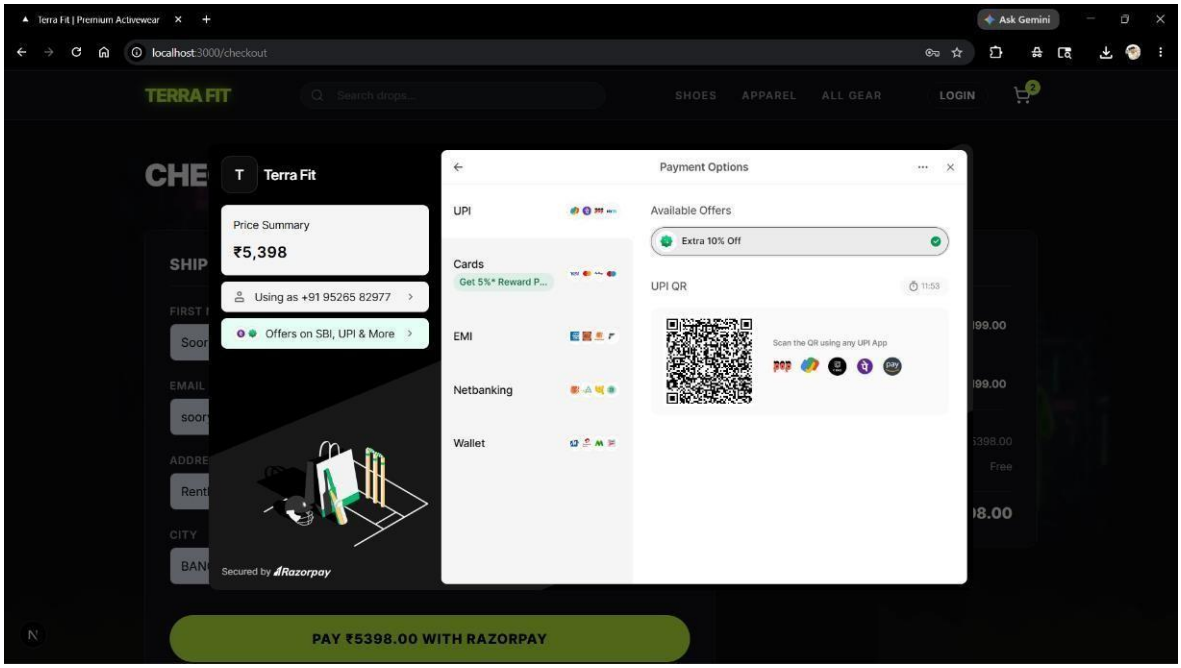
(fig:6- Contact Us)



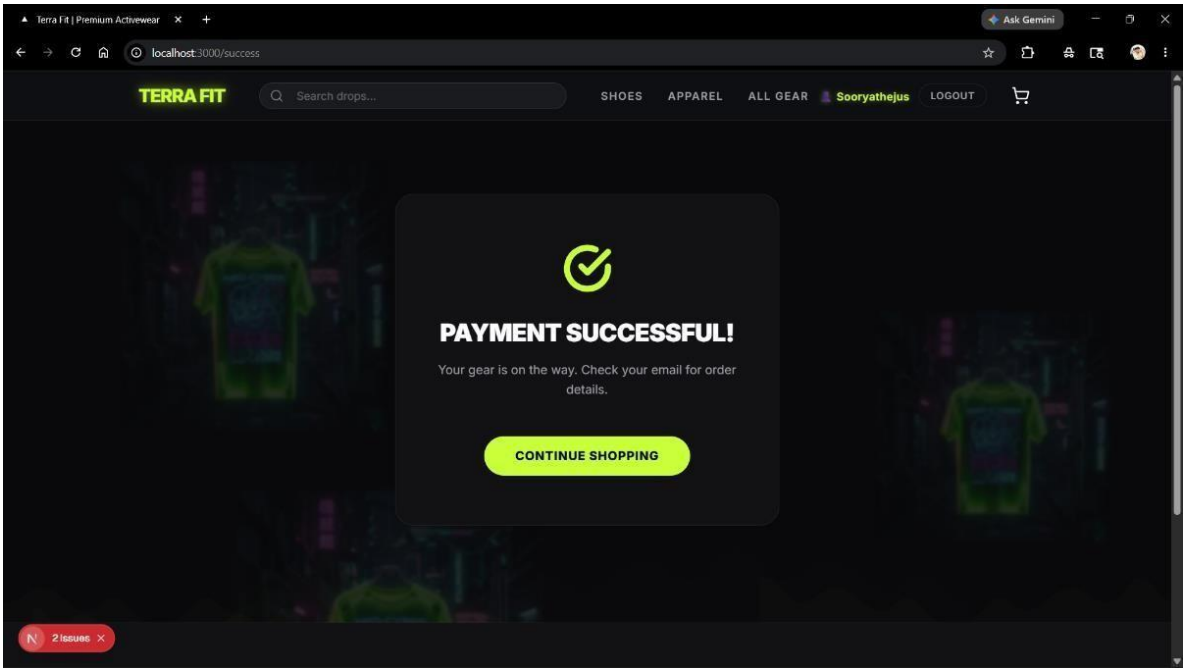
(fig:7- FAQ)



(fig:8- Checkout)



(fig:9- Payment Page)



(fig:10- Order confirmation)

9. CONCLUSION

The Terra Fit Premium Clothing Store offers a modern and efficient approach to online retail by providing a complete full-stack eCommerce solution built entirely on open-source technologies. By automating the shopping experience — from product browsing through secure payment processing — it eliminates the limitations of physical retail, reduces manual effort, and delivers a seamless purchasing journey for customers. The system ensures accurate order management and improves overall business productivity without the recurring costs of third-party SaaS platforms.

The proposed system integrates a powerful technology stack comprising Next.js 16, Node.js, Express, MongoDB, and Razorpay to provide a reliable, secure, and scalable online shopping platform. It enables real-time product browsing, efficient cart and order management, and secure payment processing through HMAC-verified Razorpay transactions. JWT-based authentication and bcrypt password hashing ensure data security and user privacy, while the responsive neon-themed interface enhances the shopping experience and helps the brand build a strong and distinctive digital identity.

Although the system may face challenges such as local database dependency, the absence of a built-in admin dashboard, and the limitation of Razorpay to Indian payment methods only, its advantages far outweigh these limitations. With proper cloud deployment using Vercel and MongoDB Atlas, along with planned future enhancements such as an admin panel, product reviews, wishlist functionality, and AI-powered recommendations, Terra Fit serves as a practical, scalable, and effective solution for modern eCommerce in the competitive streetwear market.

10.FUTURE ENHANCEMENTS

The Terra Fit Premium Clothing Store can be further improved by integrating advanced technologies to enhance its performance, usability, and overall feature set. Future enhancements may include the development of a comprehensive admin dashboard that allows the store owner to manage products, view and update orders, track inventory levels, and monitor real-time sales analytics through a secure, role-based web interface. The admin panel will support full product CRUD operations including image uploads via Cloudinary, bulk product imports via CSV, and automated low-stock alerts. Integration with a React Native mobile application will allow customers to browse products, manage their cart, track orders, and complete purchases from their smartphones with real-time push notifications for order confirmations and flash sale announcements on both Android and iOS platforms.

The system can also be enhanced by migrating to a fully cloud-based infrastructure using Vercel for the frontend, Railway for the backend, and MongoDB Atlas for a scalable and fully managed database with automated backups. Adding features such as verified product reviews and star ratings, a wishlist system with back-in-stock notifications, advanced search with filters for price range, size, colour, and rating, and an automated email notification system for order confirmations and shipping updates will significantly improve the overall customer experience. Expanding payment support to include Stripe for international transactions and PayPal for global wallet-based payments, alongside a customer loyalty and rewards program with referral codes and redeemable discount points, will broaden the platform's reach and encourage long-term customer retention.

11. BIBLIOGRAPHY

- **Laudon, K. C., & Traver, C. G. (2023). E-Commerce: Business, Technology, Society. Pearson. Link: https://api.pageplace.de/preview/DT0400.9781292343211_A39745346/preview 97812923432 11_A39745346.pdf**
- **Shopify Inc. (2024). E-commerce Platform. Link: https://www.shopify.com**
- **Amazon Inc. (2024). Online Shopping System. Link: https://www.amazon.com**
- **W3Schools (2024). Web Development Tutorials. Link: https://www.w3schools.com**
- **GeeksforGeeks (2024). Programming and Development Resources. Link: https://www.geeksforgeeks.org**
- **Zamani, B., Sandin, G., & Peters, G. (2017). Life Cycle Assessment of Clothing Libraries. Link: https://doi.org/10.1016/j.jclepro.2017.05.128**
- **Armstrong, C. M. et al. (2015). Sustainable Product-Service Systems for Clothing. Link: https://doi.org/10.1016/j.jclepro.2014.10.014**
- **Pedersen, E. R. G., & Netter, S. (2015). Collaborative Consumption in Fashion Industry. Link: https://doi.org/10.1108/JFMM-01-2015-001**